Bi-Weekly Project Progress Report & Meeting Agenda

Meeting #2 - Phase 2: Implementation

Course:	03CIT4067 Cybersecurity Project	**Date:**	October 20, 2026 (Week 7)
Project Title:	Topic 15: Design and Implementation of a Web Application Security Risk Detection and Remediation Demonstration System Based on OWASP Top 10		
Team Leader:	WONG Lai Man (24810493)	**Supervisor:**	LIAM XU
Team Members:	Mustafa, Zoutao	**Meeting Phase:**	Phase 2 - Implementation

1. Meeting Agenda

1. Phase 2 Implementation Review: System Architecture & Threat Modeling.
2. Sprint 2 Deliverables Demonstration: Detection (A05) & Remediation (A10) Demo.
3. Phase 2 Report Consolidation Progress (Chapters 1-7 & APA 7th compliance).
4. Blockers, Risk Mitigation & Task Coordination.

2. Sprint 2 Review: What We Have Done

2.1 System Architecture & Threat Modeling

Decision: Adopted a hybrid architecture. OWASP Juice Shop serves as the primary target, while custom Python Flask modules address specific OWASP 2025 nuances.
Execution: Completed a comprehensive STRIDE Threat Modeling analysis, identifying critical risks (e.g., SQLi, XSS, Fail-Open conditions) and mapping them to specific remediation strategies.

2.2 Detection & Remediation Demonstration (Key Achievements)

A05: Injection: Successfully demonstrated SQL Injection (Authentication Bypass) and Stored XSS on OWASP Juice Shop. Provided clear code-level remediation (Parameterized Queries via ORM, and DOMPurify for output encoding).
A10: Mishandling of Exceptional Conditions: Developed and tested a custom Flask module (a10_demo.py). Successfully demonstrated the vulnerability (Fail-Open granting access on exception) and the remediation (Fail-Closed returning HTTP 503 and denying access).

3. Updated WBS & Progress Status

WBS ID	Task Name	Assignee	Status
2.1	Threat Modeling on Juice Shop Architecture	WONG Lai Man	Completed
2.2	Red Team Exercise & Attack Documentation (A05)	Mustafa	Completed
2.3	Develop Custom Vulnerable/Remediated Modules (A10)	Mustafa / WONG Lai Man	Completed
2.4	Red Team Operation Logs (A01-A10 Text Deliverables)	Zoutao	Completed
3.1	Draft Phase 2 Report (Chapters 1-7)	WONG Lai Man	In Progress

4. Blockers & Task Coordination

Challenge & Environmental Constraints: Zoutao encountered health issues (hospitalization) and severe network restrictions in Mainland China, preventing Docker image pulls and local environment deployment.
Mitigation & Coordination Strategy: The Team Leader (WONG Lai Man) adjusted task allocations. Local deployment was handled by Mustafa in Hong Kong, while Zoutao focused on preparing detailed written Red Team operation logs (A01-A10). Zoutao successfully submitted two text files, which are now integrated into Chapter 5 and Appendix B.

5. Action Items & Next Steps

1. Finalize the Phase 2 Report: Integrate Zoutao's Red Team logs and Threat Modeling into Chapter 5 & Appendix B with strict APA 7th formatting.
2. Conduct OWASP ZAP Automated Scan: Generate baseline vulnerability reports.
3. Prepare Phase 3 Presentation: Begin drafting the slide deck.

6. References (APA 7th Format)

1. OWASP Foundation. (2025). *OWASP Top 10:2025: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/
2. OWASP Foundation. (2023). *OWASP Threat Modeling Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

---

Technical Appendix: Custom Flask Code Implementation (A10: Fail-Open vs. Fail-Closed)

```
# Vulnerable A10 Implementation: Fail-Open Exception Handling
@app.route('/api/v1/admin/config', methods=['GET'])
def get_admin_config():
    try:
        # Simulate auth service call that fails or times out
        user_role = verify_user_token(request.headers.get('Authorization'))
        if user_role != 'ADMIN':
            return jsonify({"error": "Unauthorized"}), 403
    except Exception as e:
        # VULNERABILITY: Log exception but default to granting access (Fail-Open)
        logger.error(f"Auth check failed due to error: {str(e)}")
        pass  # Execution continues to sensitive logic!
    return jsonify({"status": "Success", "data": "Sensitive Admin System Config"}), 200

# Secure A10 Implementation: Fail-Closed Exception Handling
@app.route('/api/v1/admin/config', methods=['GET'])
def get_admin_config_secure():
    try:
        user_role = verify_user_token(request.headers.get('Authorization'))
        if user_role != 'ADMIN':
            return jsonify({"error": "Access Denied"}), 403
    except Exception as e:
        # SECURITY FIX: Fail-Closed - Immediately abort and deny access
        logger.critical(f"Auth exception encountered: {str(e)}", exc_info=True)
        return jsonify({"error": "Service Unavailable", "message": "Security verification failed. Access
denied by default."}), 503
    return jsonify({"status": "Success", "data": "Sensitive Admin System Config"}), 200
```